Imperfect-Information RL Agent Learning: Building an Adaptable Coup Bot

A dive into Reinforcement Learning, Partially Observable Environments, and navigating Imperfect Information.

The card game Coup is one of deception, deduction, and calculated risks. Players have hidden roles, claim to have roles they don't, and call out bluffs. This makes it an inherently social game that appeals to people's ability to gauge bluffs and truth-telling (as with other hidden-information card games like Poker or BS), but it poses a challenge for AI.

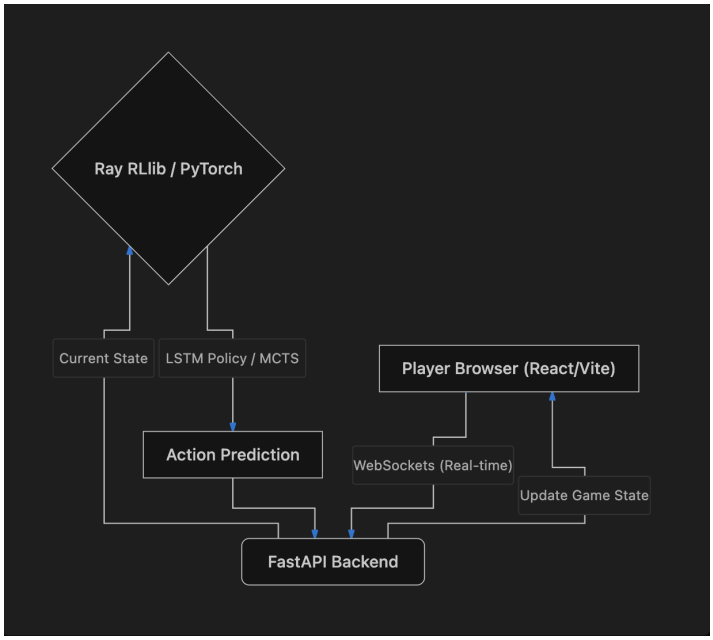
Unlike Chess or Go, where the entire board is visible (perfect information), Coup is a **Partially Observable Markov Decision Process (POMDP)**. The agent doesn't know what cards the opponents hold.

In this project, I built a Reinforcement Learning (RL) agent capable of playing Coup at a high level. The model was trained on a basic MacBook (putting its ventilation through the wringer at times) and deployed as a real-time web application where humans can play against the AI online. The following's a breakdown of how I did it, the specific engineering challenges I overcame, and the mandatory bang-my-head-against-the-wall learning moments along the way.

1. The Architecture at a Glance

Before diving into the ML specifics, it helps to understand how the final product fits together. The system is designed to be highly responsive, separating the heavy ML inference from the client-side rendering.

By containerizing this within HuggingFace, the deployment becomes reproducible and scalable.



2. Modeling the Rules of Coup

To train the AI, the rules of Coup had to be codified into a strict state machine. The game flows through several rigid phases that dictate what actions are legally possible:

1. **Start of Turn (`START_OF_TURN`)**: The active player chooses a primary action (e.g., Income, Foreign Aid, Tax, Coup, Assassinate, Steal, Exchange).
2. **Challenge Window (`ACTION_CHALLENGE`)**: If an action requires a specific role (like Tax requiring a Duke), other players have a window to call out a bluff.
3. **Block Window (`ACTION_BLOCK`)**: For blockable actions (like Foreign Aid or Steal), opponents can claim a blocking role (like Duke or Captain).
4. **Block Challenge Window (`BLOCK_RESPONSE`)**: If someone blocks, the original player can challenge that block.
5. **Reveal Phase (`REVEAL_INFLUENCE`)**: If a challenge occurs (or an Assassination/Coup succeeds), the losing player must reveal one of their hidden cards and lose that influence.
6. **Exchange Phase (`EXCHANGE`)**: If the Ambassador action succeeds, the player draws two cards and selects two to keep, returning the rest to the deck.

Structuring the environment this way allowed the neural network to learn the exact causal chain of a turn.

Here's where the model begins to deviate somewhat from a human playthrough of Coup — the real world has players spontaneously challenge or block in the middle of a round. There's no

such thing as ‘taking turns allowing/challenging’ an action in a human game; half the fun is making moves out of the blue. For the model, this was replaced with a turn-based ‘allow/challenge’ system where the order is randomized each time, so no one gets a consistent advantage.

3. Environment Design: Keeping Observation Spaces Small

To use Deep Reinforcement Learning, you have to translate the rules of the game into a mathematical environment. I used the `PettingZoo` library to build a custom multi-agent environment.

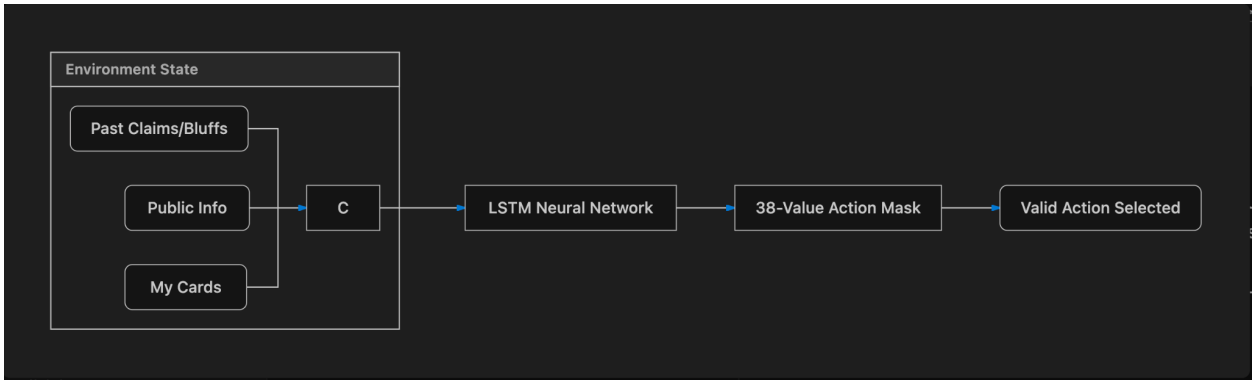
This led to my first major moment of understanding: figuring out the **observation space**.

Initially, I tried feeding the network everything it could possibly know in a highly detailed format, with matrices tracking histories in the game that a human would like to know. Betrayals, history of lies, early Duke claims, etc. However, I quickly realized that giving the network too much fluff severely slowed down training and confused the agent.

I distilled the observation space down to the bare essentials:

- **What I know:** My coins, my hidden cards.
- **What is public:** Opponents' coins, revealed dead cards.
- **The Context:** Who is the active player? What phase is the turn in?
- **The History (Crucial):** What roles have people claimed? Who has been proven *not* to have a specific role? Who holds a grudge against me?

The Solution: I flattened all of this into a **184-value vector**, paired with a **38-value Action Mask** (so the model doesn't waste time trying to take illegal actions, like launching a Coup with only 2 coins). Stripping the observation space to only what is logically necessary was the turning point that allowed the agent to start learning effectively.



4. The Pitfall of Reward Hacking

In Reinforcement Learning, the agent does exactly what you incentivize it to do. This is, surprisingly, much more frustrating than it sounds.

Early on, I tried to “help” the model learn by giving it micro-rewards:

- *"+0.1 points for successfully stealing"*
- *"+0.2 points for calling a bluff correctly"*
- *“-0.6 for getting caught in a bluff”*

And I got exactly what I wanted! The agent worked to maximize those rewards… by exchanging cards 40 times in a row to avoid taking any action that drew attention to itself. Or by stealing from a bot it *knew* had a blocking card so that it could effectively waste its turn to avoid a penalty I set up. Games drew out endlessly, illogical challenges were everywhere, and nobody actually did anything of value to the real mission of becoming good at Coup.

I won’t admit how long it took to realize micro rewards simply couldn’t replace a long clean training cycle. Eventually, I stripped away all the micro-rewards and left only the following:

- **Surviving/Winning:** Positive reward from the pot.
- **Dying:** Negative reward (-1.0).
- **Time Penalty:** A microscopic negative reward (-0.005) for every turn taken, preventing stalling.

By removing the fluff from the reward function, the model was forced to figure out the intermediate steps purely in pursuit of the ultimate goal: winning.

5. The Brain: A Hybrid AlphaZero Architecture

Because Coup requires memory and relies heavily on hidden information, standard feed-forward networks and traditional search trees fail. I implemented a hybrid architecture inspired by DeepMind's **AlphaZero**, combining an LSTM neural network with Monte Carlo Tree Search (MCTS).

The LSTM Prior

At the core of the AI is an LSTM-based neural network trained via self-play using RLlib. This network evaluates the current board state and provides two outputs:

- **Prior Probabilities (P):** A probability distribution over all legal moves, representing the network's intuition on what a good move looks like.

- **Value (V):** An evaluation of who is currently winning the game (from -1.0 to 1.0).

It also trained for 25 hours straight and made my laptop heat up more than a metal slide in the summer, but that's besides the point.

If the LSTM only played against a single strategy, it would quickly overfit and learn exactly how to exploit that one specific playstyle. I wanted a model that would use a generalizable strategy, so instead, I used Fictitious Self-Play (FSP). The agent trained against a league of past versions of itself, as well as a mix of heavily scripted heuristic bots — like a "Pathological Liar" bot that always claims whatever role it needs, or a "Rational" bot that plays purely by the numbers. This forced the agent to develop robust, well-rounded strategies that couldn't be easily broken by one specific gimmick.

Information Determinization

Because Coup is a game of hidden information, a standard search tree cannot simulate the future perfectly. Before the MCTS begins its search, it uses a technique called **Determinization**. The engine analyzes all publicly revealed cards, cross-references them with the roles each opponent is actively claiming (e.g., if a player stole coins, they claim a Captain), and deals out a simulated, consistent hand of cards to the opponents to create a "perfect information" state for the search.

PUCT Monte Carlo Tree Search

Using the determinized board state, the bot simulates the game forward hundreds of times per turn. It balances exploring new strategies and exploiting known good paths using the PUCT (Predictor Upper Confidence Bound applied to Trees) algorithm. The MCTS uses a standard AlphaZero exploration constant (`c_puct = 1.25`) to rely on proven win-rates, ensuring highly stable gameplay. To keep the web app snappy, searches are limited to 0.6 seconds. That's still hundreds of searches, and with the (albeit imperfect) LSTM as a backbone, it has a good head start to finding the next best moves.

6. Working Within Hardware Constraints

Training this complex hybrid model is computationally expensive. My hardware limit was an Apple Silicon Mac M1 Pro.

I learned quickly that Ray RLlib *is* incredibly powerful but can easily overwhelm a local machine if configured improperly. I encountered severe CPU thread contention which bottlenecked training. Here's some fixes I did for my specific situation:

1. Setting `OMP_NUM_THREADS="1"` to fix thread contention on the M1 CPU.

2. Carefully balancing the `train_batch_size` and `minibatch_size` to maximize memory bandwidth without spilling over.
3. Tuning the Proximal Policy Optimization (PPO) and entropy hyperparameters to ensure the model explored enough early on, reducing the total wall-clock time needed to reach a competent level of play.
4. Setting up periodic CRON checkups to see how model is improving over its training sequence.

7. Evaluating the MCTS Agent

To prove the agent wasn't just performing well anecdotally, I ran it through a rigorous evaluation gauntlet of 200 simulated games each against a suite of static baseline bots. In a standard game with a randomized player count of between 3 to 6, the baseline expected win rate is ~24%.

Gauntlet	**Function**	**Win Rate**	**Avg Turns**
Random	Plays randomly	88.00%	11.1
Rational	Hardcoded to play ‘rational.’ Challenge probability and lying probability change based on circumstances (1 card, cards identified in deck, etc.)	78.50%	10.5
Aggressive	Plays extensively aggressive (steal, assassinate) moves	64.50%	98.5
Honest	Never claims something they don’t have	30.50%	54.5
Mixed	A random mix of all the others	62.00%	19.2

The hybrid MCTS agent crushed the baseline expectation across the board, securing an 88% win rate against random agents and a strong 62% against a Mixed table. However, the evaluation uncovered two fascinating behavioral irregularities:

- **The Defensive Grind against Aggressive Bots (98.5 Avg Turns):** Against a table full of highly aggressive players, the agent's win rate stayed solid (64.5%), but the game length skyrocketed to nearly 100 turns. Because of the **Depth Discount Penalty** (`gamma = 0.99`), the MCTS agent learned that the optimal strategy against hyper-aggression is to play extremely passively. It essentially turtles up, takes safe actions like Income, and refuses to challenge—stalling the game until the aggressive bots eliminate each other, before cleaning up the survivors.

- **The Struggle against Honesty (30.50% Win Rate):** Funnily enough, the model struggled most against a table of exclusively Honest bots (though it's still noticeably above the baseline). The RL agent was trained in an environment where bluffing is the norm. When faced with bots that *never* bluff, the MCTS agent frequently miscalculates the risk, calling challenges on true claims and losing its influence as a result. The AI relies heavily on exploiting the deception of others, making pure honesty an oddly effective counter-strategy. It's rare to find a game of Coup where everyone else at the table is always telling the truth, but it looks like that unlikely scenario would pose problems for this model.

8. Personal Findings & Key Takeaways

- Avoid reward hacking at all costs. Models will find a way to cheat if their life depends on it (and it does)
- The model, in its current form, isn't 100% perfect. Here's some examples I've found through *extensive* playthroughs on it:
    a. Foreign Aiding with the knowledge that someone else is claiming Duke
    b. Spamming exchanges for several turns in a row (appears to work, in all honesty. Those bots end up winning since the heat stays off them and they can build the best hand when the final showdown(s) start).
    c. While endgame plays are generally completely GTO, the model struggles in the early game between racking up as many coins as possible and couping everyone or simply turtling and waiting for everyone else to kill each other.
- If I had more time and compute power, I'd definitely retrain the base LSTM model with more heuristic bots present that remain honest throughout. The honesty win rate is very poor compared to the rest in my opinion.
- If I had a scale of 1-10 (with 1 being someone who's never played Coup before and 10 being a superhuman level Coup intelligence), I would place this model at 7. There's room for improvement, but I knew when I started out that I wasn't going to build Stockfish for a game relying heavily on hidden information with just a Mac and PettingZoo. There's lots of room for experimentation, still.

Conclusion

Building the Coup RL agent was a great hands-on experience in the end-to-end Machine Learning lifecycle. It required deep reasoning about environment formulation, preventing reward exploitation, managing local hardware constraints, applying advanced search algorithms like MCTS, and finally, engineering a real-time full-stack application to serve the model to anyone in the world who wants to check it out.